



Security Assessment

QIC

Verified on September 3, 2026

SUMMARY

Project

QIC

CHAIN

BSC

METHODOLOGY

Manual & Automatic Analysis

FILES

Live Contract

DELIVERY

September 3, 2026

TYPE

Standard Audit



9

Total Findings

0

Critical

1

Major

2

Medium

3

Minor

3

Informational

0

Resolved

0 Critical

An exposure that can affect the contract functions in several events that can risk and disrupt the contract

1 Major

An opening & exposure to manipulate the contract in an unwanted manner

2 Medium

An opening that could affect the outcome in executing the contract in a specific situation

3 Minor

An opening but doesn't have an impact on the functionality of the contract

3 Informational

An opening that consists information but will not risk or affect the contract

0 Resolved

ContractWolf's findings has been acknowledged & resolved by the project

STATUS
⚠️ REVIEW REQUIRED

TABLE OF CONTENTS | QIC

| Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

| Project Information

Token/Project Details
Inheritance Graph
Call Graph

| Findings

Issues
SWC Attacks
CW Assessment
Fixes & Recommendation
Audit Comments

DISCLAIMER | QIC

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

ContractWolf's presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

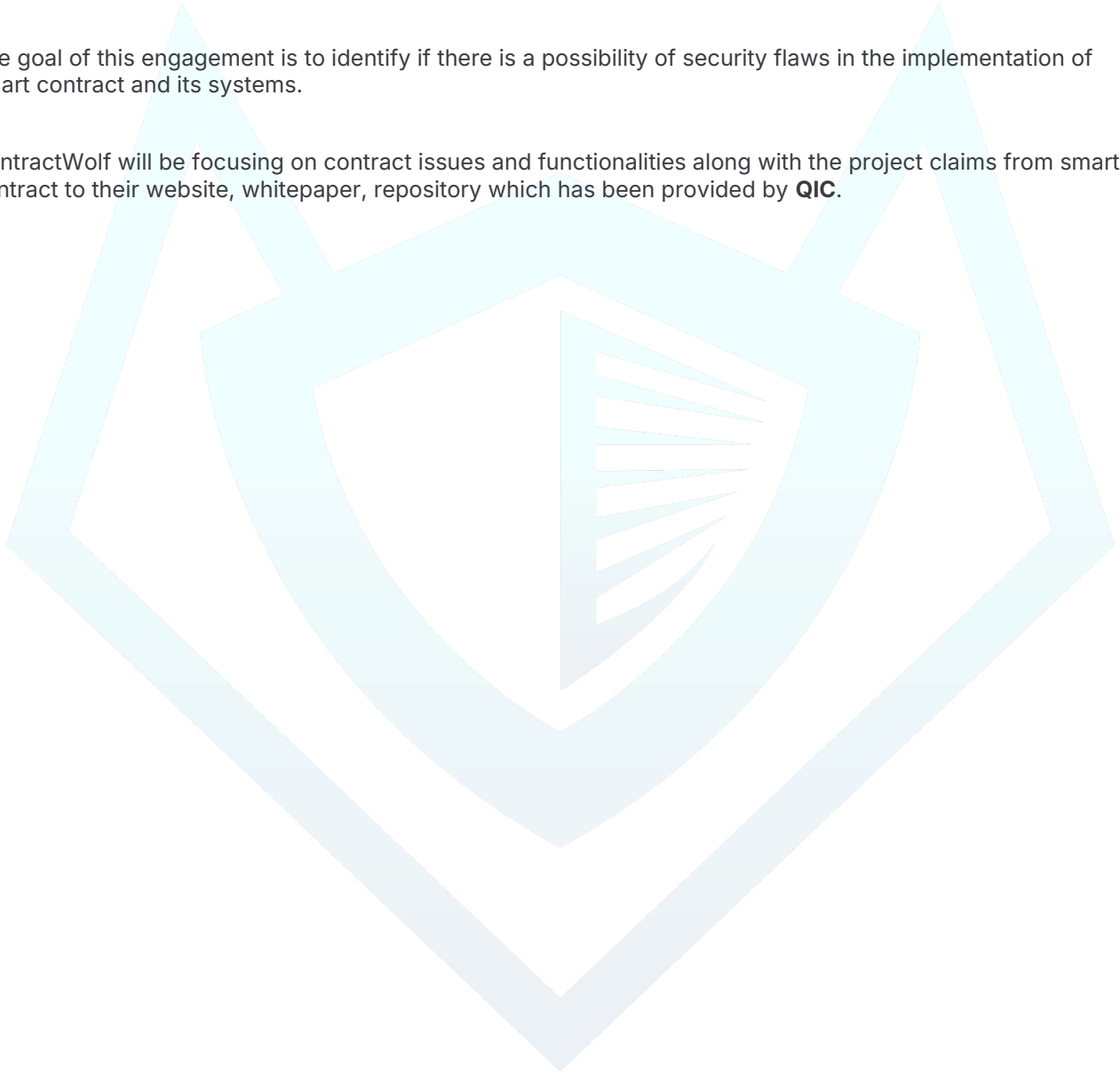
Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | QIC

QIC team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **QIC**.



AUDITING APPROACH | QIC

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
 - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
 - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.
2. Testing and automated analysis that includes :
 - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.
3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.
4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

TOKEN DETAILS | QIC



N/A

Token Name

QIC

Symbol

QIC

Decimal

18

Total Supply

1,000,000,000

Chain

BSC

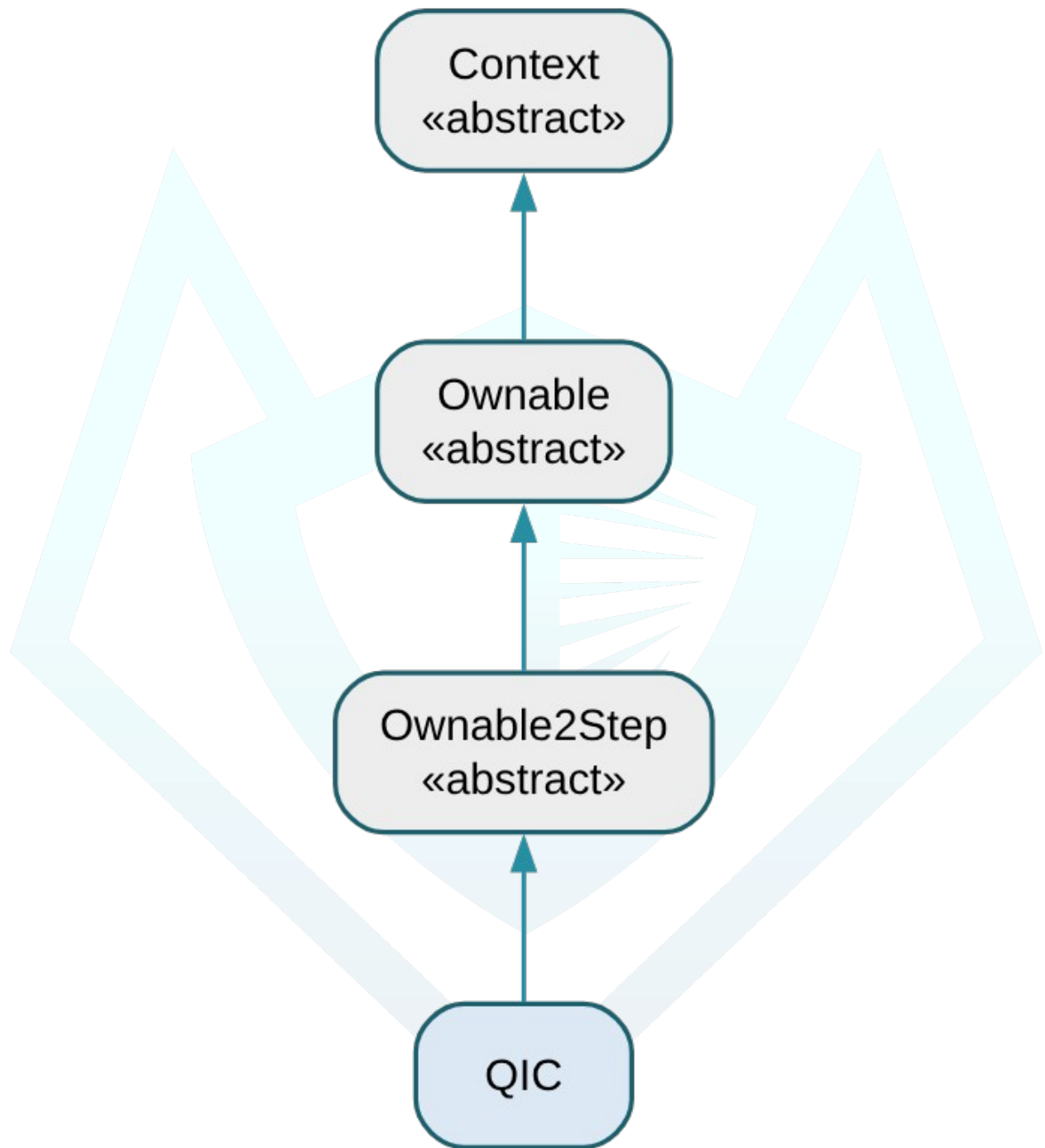
SOURCE

Source

Live Contract (0xA622cF80032808aAC02DF188a74e0B55AC038ff6)

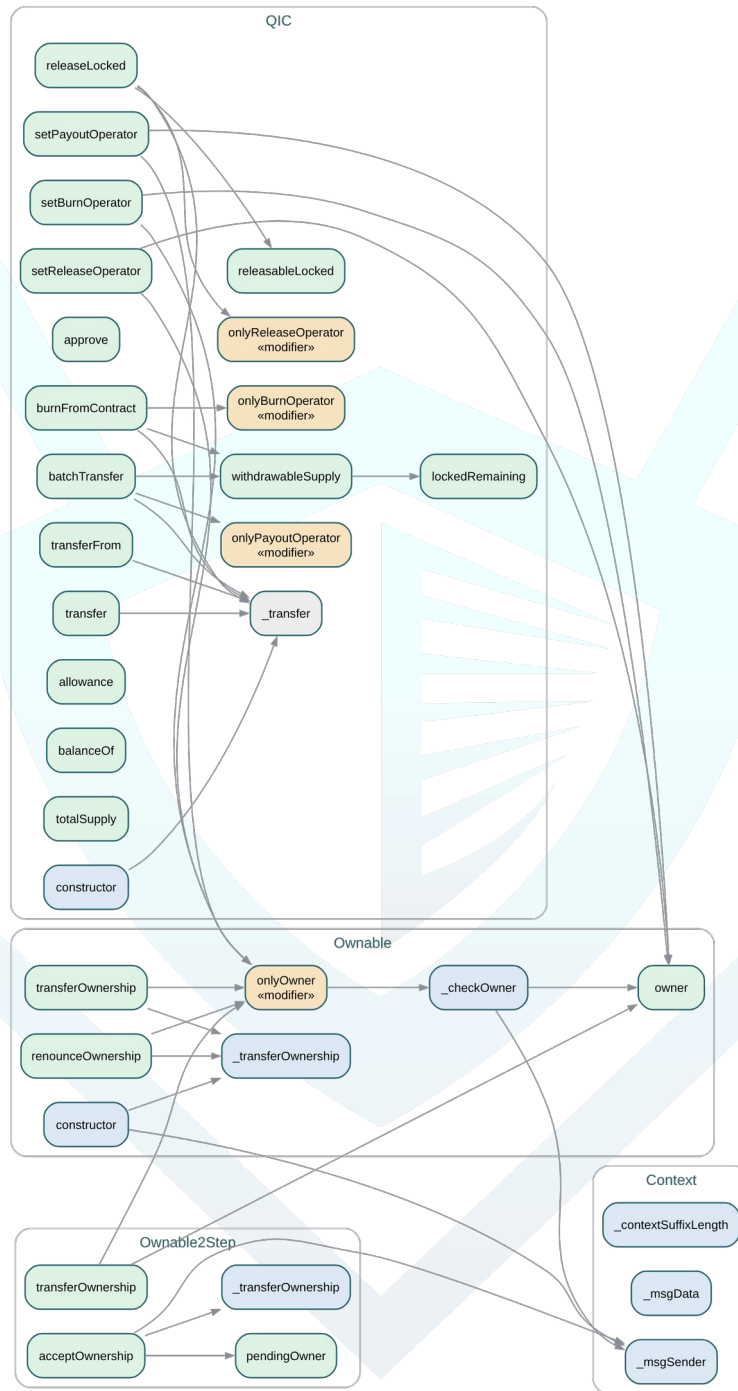
INHERITANCE GRAPH | QIC

Inheritance Graph of Contract Functions



CALL GRAPH | QIC

Call Graph of Contract Functions



FINDINGS | QIC



9 Total Findings 0 Critical 1 Major 2 Medium 3 Minor 3 Informational 0 Resolved

This report has been prepared to state the issues and vulnerabilities for QIC through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	File & Line #	Severity	Status
N/A	Locked supply releases with no waiting period	QIC.sol, L: 36, 534-565	Major	● Pending
N/A	Spend caps limit one call, not one transaction	QIC.sol, L: 47, 51, 54, 461-525, 586-619	Medium	● Pending
N/A	Owner can quietly become an operator	QIC.sol, L: 272-348	Medium	● Pending
N/A	Renouncing ownership freezes the operators in place	QIC.sol, L: 7; Ownable.sol, L: 61	Minor	● Pending
N/A	Allowances can be spent twice while being lowered	QIC.sol, L: 370-391	Minor	● Pending

N/A	Compiler version is left floating	QIC.sol, L: 2; Ownable.sol, L: 4; Ownable2Step.sol, L: 4; Context.sol, L: 4	Minor	● Pending
N/A	Burning never reduces the reported total supply	QIC.sol, L: 586-619	Informational	● Pending
N/A	Tokens sent to the contract become operator money	QIC.sol, L: 436-451, 625-656	Informational	● Pending
N/A	Token standard is implemented by hand, unchecked	QIC.sol, L: 4-7	Informational	● Pending

SWC ATTACKS | QIC

Smart Contract Weakness Classification and Test Cases

ID	Description	Status
SWC-100	Function Default Visibility	● Passed
SWC-101	Integer Overflow and Underflow	● Passed
SWC-102	Outdated Compiler Version	● Passed
SWC-103	FloatingPragma	● Not Passed
SWC-104	Unchecked Call Return Value	● Passed
SWC-105	Unprotected Ether Withdrawal	● Passed
SWC-106	Unprotected SELF DESTRUCT Instruction	● Passed
SWC-107	Reentrancy	● Passed
SWC-108	State Variable Default Visibility	● Passed
SWC-109	Uninitialized Storage Pointer	● Passed
SWC-110	Assert Violation	● Passed
SWC-111	Use of Deprecated Solidity Functions	● Passed
SWC-112	Delegate call to Untrusted Callee	● Passed
SWC-113	DoS with Failed Call	● Passed
SWC-114	Transaction Order Dependence	● Not Passed
SWC-115	Authorization through tx.origin	● Passed
SWC-116	Block values as a proxy for time	● Passed
SWC-117	Signature Malleability	● Passed
SWC-118	Incorrect Constructor Name	● Passed
SWC-119	Shadowing State Variables	● Passed
SWC-120	Weak Sources of Randomness from Chain Attributes	● Passed
SWC-121	Missing Protection against Signature Replay Attacks	● Passed
SWC-122	Lack of Proper Signature Verification	● Passed

ID	Description	Status
SWC-123	Requirement Violation	● Passed
SWC-124	Write to Arbitrary Storage Location	● Passed
SWC-125	Incorrect Inheritance Order	● Passed
SWC-126	Insufficient Gas Griefing	● Passed
SWC-127	Arbitrary Jump with Function Type Variable	● Passed
SWC-128	DoS With Block Gas Limit	● Passed
SWC-129	Typographical Error	● Passed
SWC-130	Right-To-Left-Override control character(U+202E)	● Passed
SWC-131	Presence of unused variables	● Passed
SWC-132	Unexpected Ether balance	● Passed
SWC-133	Hash Collisions With Multiple Variable Arguments	● Passed
SWC-134	Message call with hardcoded gas amount	● Passed
SWC-135	Code With No Effects	● Passed
SWC-136	Unencrypted Private Data On-Chain	● Passed

CW ASSESSMENT | QIC

ContractWolf Vulnerability and Security Tests

ID	Name	Description	Status
CW-001	Multiple Version	Presence of multiple compiler version across all contracts	✗
CW-002	Incorrect Access Control	Additional checks for critical logic and flow	✗
CW-003	Payable Contract	A function to withdraw ether should exist otherwise the ether will be trapped	✓
CW-004	Custom Modifier	major recheck for custom modifier logic	✓
CW-005	Divide Before Multiply	Performing multiplication before division is generally better to avoid loss of precision	✓
CW-006	Multiple Calls	Functions with multiple internal calls	✗
CW-007	Deprecated Keywords	Use of deprecated functions/operators such as block.blockhash() for blockhash(), msg.gas for gasleft(), throw for revert(), sha3() for keccak256(), callcode() for delegatecall(), suicide() for selfdestruct(), constant for view or var for actual type name should be avoided to prevent unintended errors with newer compiler versions	✓
CW-008	Unused Contract	Presence of an unused, unimported or uncalled contract	✗
CW-009	Assembly Usage	Use of EVM assembly is error-prone and should be avoided or double-checked for correctness	✓
CW-010	Similar Variable Names	Variables with similar names could be confused for each other and therefore should be avoided	✗
CW-011	Commented Code	Removal of commented/unused code lines	✓
CW-012	SafeMath Override	SafeMath is no longer needed starting with Solidity v0.8+. The compiler now has built-in overflow checking.	✓

FIXES & RECOMMENDATION

Locked supply releases with no waiting period

```
// contracts/QIC.sol:36 -- reads as a daily release across two years
uint256 public constant RELEASE_COUNT = 730;

// contracts/QIC.sol:534 -- the only guard on the whole schedule
require(releasedCount < RELEASE_COUNT, "QIC: all released");

// contracts/QIC.sol:551 -- and this same call is what moves it forward
releasedCount++;
releasedLocked += amount;
_transfer(address(this), LOCK_RECIPIENT, amount);
```

One percent of the supply, 10,000,000 QIC, is presented as locked and paid out over 730 releases. Nothing in the contract looks at the clock. The only thing standing between one release and the next is a counter that the release call itself increments, so the schedule advances as fast as someone is willing to call it.

The release operator can run all 730 releases inside a single transaction and move the whole locked allocation to the lock recipient at one moment. We confirmed this against a local deployment: 730 calls in one transaction, no time elapsed, and the lock recipient finished holding the full 10,000,000 QIC with nothing left locked.

Work out how many releases are due from the time that has actually passed rather than from how many times the function has been called. Record the start of the lock when the contract is deployed, decide how long one release step should take, and refuse a release that is ahead of that schedule. The existing handling of the final release, which sweeps the rounding remainder, can stay exactly as it is.

Spend caps limit one call, not one transaction

```
// contracts/QIC.sol:47 -- reads as a ceiling on how much can be burned
uint256 public constant MAX_BURN_PER_CALL = 1_000_000 ether;

// contracts/QIC.sol:598 -- but it is only ever checked against one call
require(amount <= MAX_BURN_PER_CALL, "QIC: burn limit");

// contracts/QIC.sol:502 -- the payout path caps each item the same way
require(amounts[i] <= MAX_PAYOUT_PER_RECIPIENT, "QIC: recipient amount too
large");
```

The burn and payout paths both carry limits that look like spending ceilings: a million QIC per burn, half a million per payout recipient, two hundred recipients per batch. Each is checked once, against a single call, and nothing counts how many calls are made.

A contract controlled by an operator can call these in a loop and clear every limit in one transaction. We burned fifty times the advertised per-call cap, 50,000,000 QIC, in a single transaction against a local deployment. The payout path behaves the same way, which puts the entire 980,000,000 QIC production pool within reach of one transaction if an operator key is misused or stolen.

Count what has been spent against a rolling period rather than against a single call, so the limit still means something after the tenth call. Keep a running total and the time the current period began, reset it when the period rolls over, and reject anything that would push the total past the ceiling. Apply the same treatment to the payout path so the production pool has a real daily throughput limit.

Owner can quietly become an operator

```
// contracts/QIC.sol:283 -- the owner is kept out of the operator seat
require(operator != owner(), "QIC: owner cannot payout");

// Ownable2Step.sol:35 -- but ownership can move to whoever already sits there
function transferOwnership(address newOwner) public virtual override onlyOwner
{
    _pendingOwner = newOwner;
}

// Ownable2Step.sol:52 -- the operator accepts, and the separation is gone
function acceptOwnership() public virtual {
    require(pendingOwner() == _msgSender(), "Ownable2Step: caller is not the
new owner");
}
```

Each of the three operator setters refuses to appoint the owner, which reads as a deliberate split between the key that appoints operators and the keys that spend. That check runs once, when the operator is appointed, and is never looked at again.

Ownership can simply move the other way. The owner hands ownership to an address that is already an operator, that address accepts, and now one key both spends and controls who else may spend. We confirmed both halves against a local deployment: appointing the owner as payout operator is refused, and the ownership route reaches the same place with nothing standing in the way.

Check the rule where it matters rather than only where the operator is appointed. Repeat it inside the operator gates, so a key that is both owner and operator is refused at the moment it tries to spend, and refuse an ownership transfer to an address that currently holds any of the three operator roles.

Renouncing ownership freezes the operators in place

```
// contracts/QIC.sol:7 -- inherited as-is, never overridden
contract QIC is Ownable2Step {

// Ownable.sol:61 -- leaves the contract with no owner at all
function renounceOwnership() public virtual onlyOwner {
    _transferOwnership(address(0));
}

// contracts/QIC.sol:272 -- and these are the only way to replace an operator
function setPayoutOperator(address operator) external onlyOwner {
```

The contract inherits the standard ownership module without changing it, so the owner can give up ownership at any time. Appointing operators is an owner-only action, and it is the only way to replace one.

Renouncing therefore does not remove control, it fixes it. Whichever three addresses hold the payout, burn and release roles at that moment hold them permanently. A key that is later lost or stolen can never be swapped out, and the function it controls is either stuck or in the wrong hands for good. This matters because renouncing ownership is often announced as a step towards decentralisation, and here it would do close to the opposite.

Override the renounce function so it refuses to run. Ownership should only ever move to another address through the two-step transfer the contract already inherits, which keeps the ability to rotate an operator key available for as long as the token exists.

Allowances can be spent twice while being lowered

```
// contracts/QIC.sol:382 -- replaces the old allowance outright
_allowances[msg.sender][spender] = amount;

// contracts/QIC.sol:404 -- and the spender may draw on it at any moment
require(allowed >= amount, "QIC: allowance");
_allowances[from][msg.sender] = allowed - amount;

// No increaseAllowance or decreaseAllowance exists anywhere in the contract,
// so lowering an approval always means overwriting it.
```

Approving a spender replaces whatever allowance was there before, and the contract offers no other way to adjust one. Someone lowering an approval has to send a transaction that overwrites the old figure.

A spender watching for that transaction can spend the old allowance before the change is mined and then spend the new one afterwards, taking both. This is a long-standing weakness of the approval design rather than a mistake unique to this token, but the usual ways of avoiding it are missing here, so holders have no safe route to reduce an approval.

Leave the existing approval function alone so wallets and exchanges keep working, and add the two familiar helpers that raise and lower an allowance by an amount rather than replacing it. A holder can then reduce an approval without ever passing through a moment where the old and new figures are both spendable.

Compiler version is left floating

```
// contracts/QIC.sol:2 -- any 0.8.24 or newer will do
pragma solidity ^0.8.24;

// _openzeppelin/contracts/access/Ownable.sol:4 -- and these accept far more
pragma solidity ^0.8.0;

// Deployed and verified with v0.8.24+commit.e11b9ed9.
// Nothing in the source records that.
```

The token contract accepts any compiler from 0.8.24 upwards, and the three OpenZeppelin files it builds on accept anything from 0.8.0 upwards. Both are open-ended.

The contract on chain was built with 0.8.24, but a later rebuild from this same source could pick a newer compiler and produce different bytecode. Anyone checking the deployed contract against the published source has no way to know from the source alone which compiler to use, which makes the verification harder to reproduce than it needs to be.

Name the exact compiler version in every file rather than a range, so that rebuilding from this source produces the same bytecode that is deployed and verified today.

Burning never reduces the reported total supply

```
// contracts/QIC.sol:57 -- burning means sending to a dead address
address public constant BLACKHOLE = 0x00000000000000000000000000000000dEaD;

// contracts/QIC.sol:609 -- the tokens move, they are not destroyed
_transfer(address(this), BLACKHOLE, amount);

// contracts/QIC.sol:214 -- so this keeps returning the original figure
return _totalSupply;
```

Burning here means sending tokens to a dead address rather than destroying them, and the contract says so plainly in its own comments. The consequence is that total supply keeps reporting 1,000,000,000 QIC no matter how much has been burned.

This is a deliberate choice and it is applied consistently. The risk is in how it is read elsewhere: explorers, price aggregators and anything working out circulating supply from total supply will overstate it by exactly the amount sitting at the dead address.

Keep the mechanism and make the number easier to read. Add a view that reports supply with the dead address and the contract's own holdings taken out, and leave total supply as the plain token figure. Reading the dead address balance rather than the internal counter also picks up tokens that holders send there themselves.

Tokens sent to the contract become operator money

```
// contracts/QIC.sol:441 -- the spendable pool is just the contract's balance
uint256 balance = _balances[address(this)];
return balance > locked ? balance - locked : 0;

// contracts/QIC.sol:637 -- nothing stops the contract being a destination
require(to != address(0), "QIC: zero recipient");

// contracts/QIC.sol:492 -- and a batch may name it as a recipient too
require(recipients[i] != address(0), "QIC: zero recipient");
```

The pool that the payout and burn operators may spend from is worked out from whatever the contract happens to hold, less the part still locked. Any holder can send QIC to the contract address, and doing so enlarges that pool.

Sending tokens to a contract address by mistake is a common error, and here those tokens are quietly absorbed into the operators' spendable balance with no way for the sender to get them back. Separately, a batch payout will accept the contract's own address as a recipient, which does nothing except cost gas and record a transfer from the contract to itself.

Decide whether tokens sent in should be recycled into the pool. If they should not, refuse the contract's own address as a transfer destination, which also removes the pointless batch case. If they should, say so somewhere holders will see it, because at the moment the behaviour is only visible by reading the code.

Token standard is implemented by hand, unchecked

```
// contracts/QIC.sol:4 -- Ownable2Step already brings Ownable in with it
import "@openzeppelin/contracts/access/Ownable2Step.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

// contracts/QIC.sol:7 -- no token interface here, so nothing is checked
contract QIC is Ownable2Step {

// contracts/QIC.sol:209 -- every standard function is written out by hand
function totalSupply() external view returns (uint256) {
```

The token functions are written out by hand rather than inherited, and the contract does not declare that it implements the token interface. The signatures are correct as they stand and the deployed contract works with wallets and exchanges as expected.

What is missing is the safety net. Because the interface is never declared, the compiler is not checking the implementation against it, so a future edit that changed one of these signatures would compile without complaint. The second import line is also redundant, since the module above it already pulls that file in.

Declare the token interface on the contract so the compiler checks the implementation against it on every build, and drop the redundant import. Declaring the interface means removing the two event declarations the contract makes for itself, because the interface already declares them, and marking the six standard functions as overrides.

AUDIT COMMENTS | QIC

Smart Contract audit comment for a non-technical perspective

- The release operator can release the entire 10,000,000 QIC locked allocation immediately. Despite the 730-step schedule, the contract enforces no waiting period between releases.
- The burn operator can send the whole 980,000,000 QIC production pool to the dead address. The 1,000,000 per-call cap limits one call, not how many calls are made in a transaction.
- The payout operator can distribute the entire 980,000,000 QIC production pool to any addresses it chooses, in unlimited batches.
- The owner can replace the payout, burn and release operators at any time, without notice or delay.
- The owner can transfer ownership to an address that already holds an operator role, placing operator authority and owner authority in a single key.
- Burning does not reduce total supply. The token will always report 1,000,000,000 QIC total supply no matter how much is sent to the dead address.
- QIC sent to the contract address by mistake cannot be recovered by the sender and becomes spendable by the payout and burn operators.
- The owner cannot mint new tokens. Total supply is fixed at 1,000,000,000 QIC and no minting function exists anywhere in the contract.
- The owner cannot pause transfers, blacklist wallets, or set buy/sell taxes or fees. None of these mechanisms exist in the contract.
- The owner cannot take tokens out of a holder's wallet. There is no forced-transfer or seize path, and transfers require the holder's own signature or an allowance they granted.
- The 10,000,000 QIC locked allocation cannot be paid out or burned. Both the payout and burn paths reserve it, and this was verified to hold once the production pool is exhausted.
- Ownership transfer is two-step, so ownership cannot be lost by sending it to a wrong or unreachable address.
- The lock recipient and the direct allocation recipient are fixed constants in the contract and cannot be changed after deployment.
- The contract cannot receive or hold BNB. It has no payable function, so no BNB can become stuck in it.



CONTRACTWOLF

Blockchain Security - Smart Contract Audits